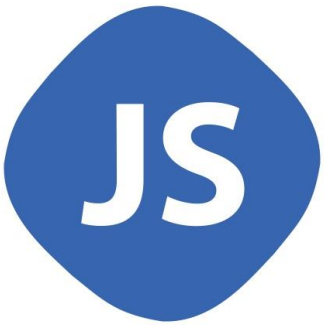


Атрибуты и стили 1

- Введение
- Свойства для работы с атрибутами
- Свойство attributes

Введение

DOM-интерфейс позволяет работать как со свойствами элементов, так и с их атрибутами.

DOM-свойства

Когда браузер загружает страницу, он парсит HTML-код и генерирует из него DOM-объекты. Все элементы страницы станут DOM-объектами со своими **встроенными свойствами**, которые учитываются браузером при построении дерева визуализации. Узел, соответствующий тегу **p** имеет один набор свойств, узел **div** – другой набор, узел **#text:** – третий.

На прошлом уроке мы рассмотрели **встроенные DOM-свойства**. Их достаточно много.

С другой стороны. DOM-узлы – это обычные **объекты JavaScript**. Объекты обладают свойствами, которые можно изменять, технически нас никто не ограничивает, разработчик может добавить своё **собственное свойство** к объекту.

example_1. Пользовательское свойство элемента

```
<script>
      // ссылка на элемент div
      let div =
      document.body.firstElementChild.nextElementSibling.nextElementSib
```



```
        ling;
        // назначили собственное свойство элементу
        div.mytitle = "Блок цитат Владимира Набокова"
        // встроенное свойство
        console.log(div.nodeName);
        // встроенное свойство
        console.log(div.textContent);
        // пользовательское свойство
        console.log("--> ", div.mytitle);
</script>
```

Итак. DOM-узлы обладают:

- **встроенными** свойствами (всегда);
- **пользовательскими** (назову так) свойствами (опционально).

Нестандартные свойства видны только в JavaScript и никак не влияют на отображение соответствующего тега.

Обратите внимание, **пользовательские DOM-свойства**:

- Могут иметь любое значение.
- Названия свойств **чувствительны** к регистру.
- Работают за счёт того, что **DOM-узлы являются объектами JavaScript**.

HTML-атрибуты

У HTML тегов могут быть **атрибуты**. Когда браузер парсит HTML, чтобы создать DOM-объекты для тегов, он распознаёт **стандартные атрибуты** и создаёт для них **DOM-свойства**. Большинство стандартных HTML-атрибутов становятся свойствами соответствующих объектов.

Важно. Для узлов-элементов большинство **стандартных HTML-атрибутов автоматически** становятся **свойствами DOM-объектов**.

Пример 1:

```
<div id="person" class="card-body">...</div>
```

DOM-узел, соответствующий элементу **div**, имеет набор присущих ему **встроенных свойств** (nodeName, nodeType, tagName и др.) плюс набор свойств, полученных из **стандартных атрибутов**:

- **id** = "person"
- **class** = "card-body"

Пример 2:

```
<p id="education" class="card-title" title="Образование">...</p>
```

DOM-узел, соответствующий элементу **p**, имеет набор присущих ему **встроенных свойств** (nodeName, nodeType, tagName и др.) плюс набор свойств, полученных из **стандартных атрибутов**:

- **id** = "education"
- **class** = "card-title"
- **title** = "Образование"

example_2. Свойство из стандартного атрибута id тега div

```
<script>
    // ссылка на элемент div
    let div =
    document.body.firstElementChild.nextElementSibling.nextElementSib
    ling;
    div.mytitle = "Блок цитат Владимира Набокова"
    // встроенное свойство
    console.log(div.nodeName);
```

```
    // пользовательское свойство
    console.log(div.mytitle);
    // свойство из стандартного атрибута id тега div
    console.log(div.id);
</script>
```

Таким образом, когда у элемента есть **id** или другой **стандартный атрибут**, для него создаётся **соответствующее свойство**.

Но...

> ***Важно***. Преобразование атрибута в **свойство не происходит**, если атрибут **нестандартный**.

example_3. Нестандартный атрибут не создает свойство

```
<script>
    // ссылка на элемент div
    let div = 
    document.body.firstElementChild.nextElementSibling.nextElementSib
    ling;
    div.mytitle = "Блок цитат Владимира Набокова"
    // встроенное свойство
    console.log(div.nodeName);
    // пользовательское свойство
    console.log(div.mytitle);
    // стандартный атрибут
    console.log(div.id);
    // нестандартный атрибут
    console.dir(div.myattr);
```

```
</script>
```

Учтите, стандартный атрибут для одного тега может быть нестандартным для другого. Например, атрибут **type** является стандартным для элемента **input** (HTMLInputElement), но не является стандартным для **body** (HTMLBodyElement).

Стандартные атрибуты **описаны в спецификации для соответствующего класса элемента.**

Таким образом, для **нестандартных атрибутов не будет соответствующих DOM-свойств.** Каким образом можно получить доступ к таким атрибутам?

Все атрибуты доступны с использованием свойства **attributes** (расмотрим ниже). Либо, что предпочтительнее, с помощью следующих методов (рассмотрим в следующем уроке):

- **getAttribute**(name) – получает значение атрибута.
- **setAttribute**(name, value) – устанавливает значение атрибута.
- **hasAttribute**(name) – проверяет наличие атрибута.
- **removeAttribute**(name) – удаляет атрибут.

В отличие от свойств, атрибуты:

- Всегда являются строками.
- Их имя **нечувствительно** к регистру (ведь это HTML).
- Видны в innerHTML.

DOM-свойства типизированы

DOM-свойства из атрибутов **не всегда** являются строками. Например:

- **атрибут** style – **строка**
- **НО, свойство** style – **объект**.

В свою очередь, свойство чекбоксов input.checked имеет **логический** тип.

example_4. DOM-свойства из атрибутов не всегда строка

```
<script>
```

```
    // ссылка на элемент div
    let div = document.body.firstElementChild.nextElementSibling;
    // свойство из атрибута - строка
    console.log(div.id);
    // свойство из атрибута - объект
    console.log(div.style);
    // доступ к свойству объекта
    console.log(div.style.width);
    // свойство из атрибута - логическое значение
    let el = div.firstElementChild;
    console.log(el.checked);
</script>
```

Связь свойств с атрибутами

В DOM-интерфейсе понятия HTML-атрибутов и DOM-свойств элементов тесно связаны. При создании модели страницы для **каждого элемента создаются одноимённые с атрибутами свойства** и присваиваются им те же значения. Однако, это справедливо не для всех атрибутов.

Например:

Атрибуту **class** соотвествует свойство **className**, атрибуту **data** - свойство **dataset**.

example_5. Имя атрибута и свойства могут отличаться (хоть это и не логично)

```
<script>
    // ссылка на элемент div
    let div =
    document.body.firstElementChild.nextElementSibling.nextElementSib
    ling;
    // имя атрибута = имя свойства
```

```
    console.log(div.id);
    // имя атрибута class, имя свойства className
    console.log(div.class); // undefined
    console.log(div.className); // card
</script>
```

Когда стандартный атрибут изменяется, соответствующее свойство автоматически обновляется. Это работает и в обратную сторону (за некоторыми исключениями). Этот эффект называется **синхронизацией** и будет рассмотрен в следующем уроке.

Многие элементы HTML имеют только базовый набор атрибутов: id, class, style и title, а также языковые атрибуты lang и dir.

Свойства для работы с атрибутами

hidden

Свойство **hidden** позволяет получить или изменить атрибут **hidden**.

Синтаксис:

```
node.hidden;  // получить атрибут

node.hidden = true|false;  // установить атрибут
```

Технически, атрибут hidden работает так же, как style="display:none". Но его проще поставить через JavaScript (меньше букв), и могут быть преимущества для скринридеров и прочих нестандартных браузеров.

example_6. Скрываем элемент свойством hidden

```
<script>
    // ссылка на узел div
    let div = document.body.firstElementChild.nextElementSibling;
```

```
    // скроем элемент
    div.hidden = true;
</script>
```

id

Свойство **id** позволяет установить или возвратить идентификатор элемента (значение атрибута **id** элемента).

Синтаксис:

```
node.id  // возвращает значение
node.id = "id"  // устанавливает значение
```

где:

- **id** - идентификатор элемента.

Возвращаемое значение:

Строка, содержащая идентификатор элемента.

example_7. Стандартный атрибут id

```
<script>
    // ссылка на узел body
    let body = document.body;
    let header = body.firstElementChild;
    let div = header.nextElementSibling;
    // поменяем атрибут id
    div.id = "plus";
    // к узлу применились настройки из <style>
    // для идентификатора id
    console.log("Текущий идентификатор элемента: id =", div.id);
```

```
</script>
```

className

Свойство **className** устанавливает или возвращает значение атрибута **class**.

Синтаксис:

```
node.className; // возвращает значение

node.className = "name";  // устанавливает значение
```

где:

- **name** - задает имя класса элемента. Чтобы применить несколько классов, необходимо разделить их пробелами. Например:
  `node.className = "test demo";`

Возвращаемое значение:

Строка, представляющая класс, или разделенный пробелами список классов, элемента.

Данное свойство удобно использовать, если элементу присвоен только один класс. Но если их несколько, то с этим свойством работать неудобно. Для таких случаев в DOM введено ещё одно свойство для работы с классами - **classList**.

example_8. Свойство className

```
<script>
      // ссылка на узел div
      let div = document.body.firstElementChild.nextElementSibling;
      // добавим в свойство className два класса
      div.className = "go plus";
      // выведем значение свойства
      console.log(div.className); // go plus
</script>
```



classList

Свойство **classList** содержит список (псевдомассив) классов элемента.

Синтаксис:

```
node.classList; // возвращает список классов
```

Для работы с этим массивом можно воспользоваться циклом (изучается в следующих уроках) или **встроенными методами**:

Метод	Описание
element.**classList.add** (class1, class2,...)	Добавляет один или несколько имен классов элемента. Если указанный класс уже существует, не будет добавлен
element.**classList.remove** (class1, class2,...)	Удаляет один или несколько имен классов из элемента.
element.**classList.contains** (class)	Проверяет, имеется ли класс (true) или нет (false)
element.**classList.toggle** (class)	Добавляет класс, если его нет, или удаляет его, если он уже есть

example_9. Добавление классов в classList

```
<script>
	// ссылка на узел div
	let div = document.body.firstElementChild.nextElementSibling;
	// добавим class узлу
	div.classList.add("border");
	// и еще один
	div.classList.add("font");
</script>
```

example_10. Добавление и удаление классов

```
<script>
      // ссылка на узел div
      let div = document.body.firstElementChild.nextElementSibling;
      / добавим узлу class="border"
      div.classList.add("border");
      // добавим узлу class="font"
      div.classList.add("font");
      // удалим class="go"
      div.classList.remove("go");
      // проверим наличие классса с именем "go"
      if (div.classList.contains("go"))
            console.log("go class detected");
      else console.log("go class not found");
</script>
```

style

Свойство **style** возвращает объект, который дает доступ к **стилю элемента** на чтение и запись.

- С помощью свойства style можно изменять **большинство CSS-свойств**.

Например:

node.style.width="100px" работает так, как будто у элемента в атрибуте прописано style="width:100px".

- Для свойств, названия которых состоят из нескольких слов, используется так называемая **camelCase** (верблюжья) нотация:

Например:

background-color => element.**style.backgroundColor**



```
z-index => element.style.zIndex
border-left-width => element.style.borderLeftWidth
```

- Чтобы сбросить поставленный стиль, в **style** присваивают пустую строку: element.style.width="". При сбросе свойства style стиль будет взят из CSS.

Например:

Чтобы скрыть элемент, можно присвоить: element.style.display = "none". Чтобы показать его обратно – необязательно явно указывать другой display, наподобие element.style.display = "block". Можно просто снять поставленный стиль: element.style.display = "".

- Свойство **style** является **специальным объектом**, ему нельзя присваивать строку.

Например:

node.style="color:blue"; - запись работать не будет.

example_11. Style возвращает объект, который дает доступ к стилю элемента

```
<script>
      // ссылка на узел div
      let div = document.body.firstElementChild.nextElementSibling;
      // выведем объект style
      console.log(div.style);
      // изменим некоторые CSS-свойства узла
      div.style.backgroundColor = "darkgrey";
      div.style.color = "#fff";
      div.style.border = "2px dotted red";
      div.style.padding = "20px";
</script>
```





cssText

Свойство **cssText** позволяет задать CSS стили массово одной строкой. При этом все содержимое атрибута **style перезаписывается**.

Синтаксис:

```
node.style.cssText; // возвращает значение

node.style.cssText = "свойство 1: значение; свойство 2: значение...";
// устанавливает одно или несколько значений
```

Злоупотребление этим свойством может привести к ошибкам по той причине, что **cssText** затирает все из атрибута **style**, это значит, что все ранее установленные стили CSS просто затрутся.

example_12. Массовое задание стилей

```
<script>
	// ссылка на узел body
	let div = document.body.firstElementChild.nextElementSibling;
	//  перезапишем настройки класса go
	div.style.cssText = "background: #fff; border: 1px solid green;
	color: #000; font-family: Verdana; font-size: 0.9em";
</script>
```

Свойство attributes

У любого элемента есть свойство **attributes**, с помощью которого можно получить коллекцию его атрибутов, в виде объекта **NamedNodeMap**. Каждый атрибут в этой коллекции имеет имя, которое совпадает с именем атрибута. Доступ к атрибуту в этой коллекции осуществляется по его **индексу** или с помощью метода **item**. Отсчёт атрибутов в коллекции начинается с 0.

attributes



Свойство **attributes** возвращает коллекцию атрибутов указанного узла в качестве объекта **NamedNodeMap**. Узлы могут быть доступны **с помощью индексов** (индекс начинается с 0).

Синтаксис:

```
node.attributes; // возвращает объект NamedNodeMap
node.attributes[index]; // возвращает атрибут по индексу
```

Параметр:

- **index** - индекс узла атрибута в NamedNodeMap.

У любого атрибута есть свойства **name** и **value**, с помощью которых можно получить имя атрибута и его значение:

```
node.attributes[index].name; // возвращает имя атрибута
node.attributes[index].value; // возвращает значение атрибута
node.attributes[index].value = "value"; // устанавливает значение атрибута
```

example_13. Свойство attributes и коллекция NamedNodeMap

```
<script>
	// ссылка на узел body
	let body = document.body;
	let div = body.firstElementChild.nextElementSibling;
	console.log("Объект NamedNodeMap: ", div.attributes);
	// первый атрибут
	console.log("Первый атрибут: ", div.attributes[0]);
	// второй атрибут
	console.log("Второй атрибут: ", div.attributes[1]);
	console.log("Имя второго атрибута: ", div.attributes[1].name);
	console.log("Значение второго атрибута: ",
	div.attributes[1].value);
```

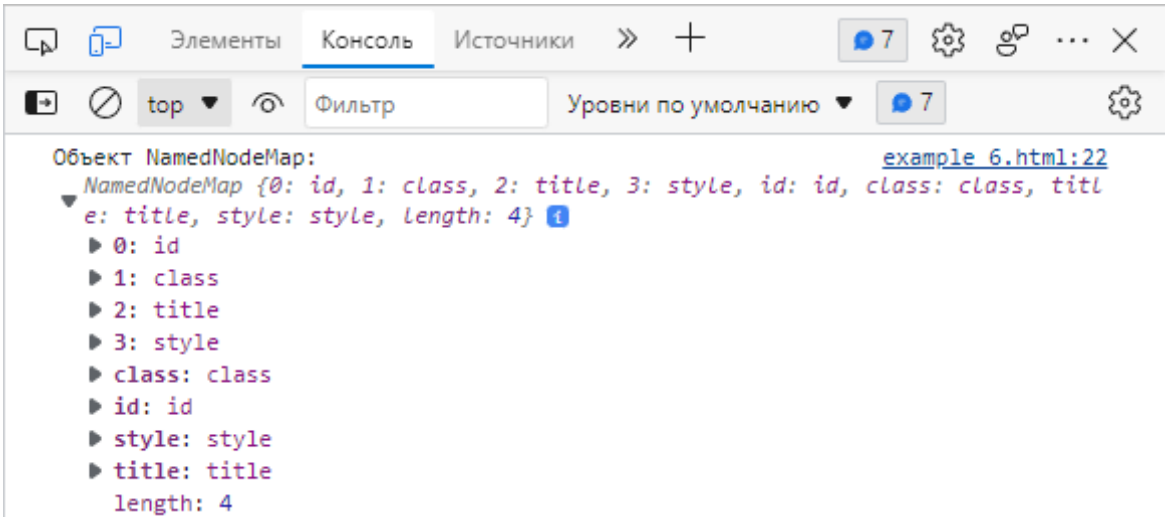


```
</script>
```

В примере example_13 обратите внимание на объект **NamedNodeMap**, который имеет вид:

```
{0: id, 1: class, 2: style, id: id, class: class, style: style,
length: 3}
```

> ***Важно***. Объект **NamedNodeMap** пригодится нам в примерах ниже.



item

Метод **item** () возвращает узел атрибута по указанному индексу.

Синтаксис:

```
node.attributes.item(index); // возвращает атрибут по индексу

node.attributes[index]; // альтернативная форма записи
```

Параметр:

- **index** - индекс узла атрибута в NamedNodeMap.

example_14. Метод attributes.item()

```
<script>
      // ссылка на узел div
      let div = document.body.firstElementChild.nextElementSibling;
      console.log("Коллекция атрибутов узла: ", div.attributes);
      // доступ к первому атрибуту
      console.log(div.attributes.item(1));
      // или так...
      console.log(div.attributes[1]);
      console.log("Имя третьего атрибута: ",
      div.attributes.item(2).name);
      console.log("Значение третьего атрибута: ",
      div.attributes.item(2).value);
</script>
```

length

Свойство **length** возвращает количество узлов-атрибутов в **NamedNodeMap**.

Синтаксис:

```
node.attributes.length; // возвращает количество атрибутов в коллекции
```

example_15. Количество узлов-атрибутов в коллекции NamedNodeMap

```
<script>
      // ссылка на узел div
      let div = document.body.firstElementChild.nextElementSibling;
      // вывод свойства length
      console.log (div.attributes.length); // 4
</script>
```

Свойство **attributes** предполагает, что обращение к атрибутам узла будет выполняться **по его индексу**. Это не всегда удобно. Для работы с **узлами**

атрибутов (получения значения узла, добавление нового узла или удаления существующего) могут использоваться следующие **методы объекта NamedNodeMap**:

- **getNamedItem**()
- **setNamedItem**()
- **removeNamedItem**()
- **hasAttributes**()

getNamedItem

Метод **getNamedItem**() возвращает узел атрибута с указанным именем из объекта **NamedNodeMap**.

Синтаксис:

```
node.attributes.getNamedItem(nameName); // возвращает атрибут по имени
```

Параметр:

- **nodeName** - имя узла в **NamedNodeMap**.

Возвращаемое значение:

Объект узел, представляющий узел атрибута с указанным именем.

example_16. Метод getNamedItem()

```
<script>
      // ссылка на узел div
      let div = document.body.firstElementChild.nextElementSibling;
      // набор -> NamedNodemap
      // console.log(div.attributes);
      // получим узел атрибута с именем id
      console.log(div.attributes.getNamedItem("id"));
      console.log(div.attributes[0]); // тот же узел, но через индекс
```

```
    // получим узел атрибута с именем title
    console.log(div.attributes.getNamedItem("title"));
    // console.log(div.attributes[1]); // тот же узел, но через индекс
    // получим узел атрибута с именем onclick
    console.log (div.attributes.getNamedItem("onclick"));
    // console.log(div.attributes[2]); // тот же узел, но через индекс
    // получим узел атрибута с именем style
    console.log(div.attributes.getNamedItem("style"));
    // console.log(div.attributes[3]); // тот же узел, но через индекс
</script>
```

removeNamedItem

Метод **removeNamedItem**() удаляет узел с указанным именем в объекте **NamedNodeMap**.

Синтаксис:

```
node.attributes.removeNamedItem(nameName); // удаляет атрибут по индексу
```

Параметр:

- **nodeName** - имя узла в **NamedNodeMap** который необходимо удалить.

Возвращаемое значение:

Объект узел, представляющий удаленный узел атрибута.

example_17. Метод removeNamedItem()

```
<script>
    // ссылка на узел div
```

```
    let div = document.body.firstElementChild.nextElementSibling;
    // удалим узел-атрибут
    var attr = div.attributes.removeNamedItem("id");
    // удаленный узел-атрибут
    console.log(attr); // id='citate'
    // получим атрибут id
    console.log(div.attributes.getNamedItem("id")); // null
</script>
```

setNamedItem

Метод **setNamedItem**() добавляет указанный узел к объекту **NamedNodeMap**.

Синтаксис:

```
node.attributes.setNamedItem(node); // добавляет атрибут к коллекции
```

Параметр:

- **node** - узел для добавления или замены в **NamedNodeMap**.

Комментарий:

Если узел уже существует, то он будет **заменен**. Замененный узел будет возвращаемое значением, в противном случае возвращаемое значение будет **нулевым**.

createAttribute

Метод createAttribute() создает новый **узел атрибута** и возвращает его.

Синтаксис:

```
document.createAttribute(name); // создает атрибут
```

Параметр:

- **name** - строка, содержащая **имя** атрибута.

Возвращаемое значение:

Узел атрибута.

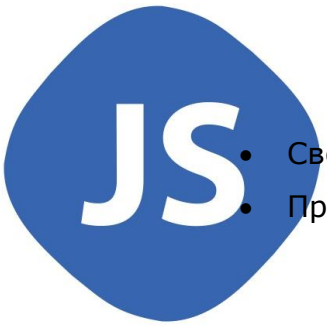
example_18. Метод setNamedItem()

```
<script>
	// ссылка на узел div
	let div = document.body.firstElementChild.nextElementSibling;
	// создаем атрибут
	var attr = document.createAttribute("id");
	// назначим атрибуту значение
	attr.value = "citate";
	div.attributes.setNamedItem(attr);
	// или так ;)
	// div.id = "citate";
</script>
```

К сведению. Очень много разных способов сделать даже самое простое действие с узлом элемента (и это еще не все). Но сейчас, на мой взгляд, важнее всего рассмотреть, **что в принципе может JS при работе с деревом DOM документа**. А навык выбора лучшего решения придет со временем и с опытом разработки.

В **самостоятельных работах** к уроку разбираются вопросы:

- Получение и назначение атрибутов блоков.
- Получение и назначение встроенных стилей.
- Стилевое оформление блоков.



- Свойства и методы узлов-атрибутов.
- Преобразование объекта в узел-элемент.

